PROJECT REPORT
BUBBLE SORT

COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE

Name: SABAHAT
Enrollment: 01-134241-041

Name: ROHAN MUNIR
Enrollment: 01-134241-040

Name: MARYAM KHALID
Enrollment: 01-134241-020

Objective:

The objective of this project was to implement the **Bubble Sort algorithm** using **Assembly Language** on the **8086** simulated by the emu8086 environment. The program is designed to accept an array of seven single-byte (0-255) numerical values from the user, sort them efficiently, and display the sorted result. This implementation demonstrates fundamental concepts in low-level programming, including direct **memory manipulation**, **register management**, and **essential I/O data conversion.**

Theoretical Background:

Bubble Sort is a simple comparison sorting algorithm. It works by repeatedly stepping through the list, comparing adjacent elements, and swapping them if they are in the wrong order. The pass through the list is repeated until the list is sorted. Each pass "bubbles up" the largest unsorted element to its correct final position. The algorithm's time complexity is O(n^2).

Type of Program:

This is a sorting and display program implemented in x86 Assembly Language (MASM/TASM).
The program collects numeric inputs from the user, sorts them, and prints the sorted list.

Data Structure Used:

The code uses the following data structures and control-flow mechanisms:

1. **Array**

array db max_elements dup(?)
Used to store the 7 user-provided numbers.

2. **Loops**
    - Outer and inner loops for Bubble Sort
    - Input loop to read 7 values
    - Print loop to display sorted values

3. **Conditional Jumps**

- *JE, JL, JNZ, JBE*, etc.
  Used to compare values and control program flow.

4. **Stack Operations**

Used inside procedures:

- PUSH / POP to save registers during:
  - *read_and_convert_byte*
  - *byte_to_ascii_print*

5. **Buffers**

- input_buffer for storing raw user input
- digit_buffer for output conversion before printing

Features / Functions

Main Features

Input Module

- Accepts a list of unsorted numbers
- Sorting Module
- Applies Bubble Sort logic
- Compares adjacent elements
- Swaps values when necessary
- Automatically repeats until the entire list is sorted

Output Module

- Displays the sorted list
- Optionally shows the number of swaps or iterations

Implementation:

The program is divided into three main functional blocks: User Input, Sorting, and Output.

User Input:

In Assembly, the main difficulty is taking numeric input because the CPU receives digits as **ASCII characters**, not real numbers. The **READ_AND_CONVERT_BYTE** procedure solves this by reading one digit at a time using **INT 21h (AH=01h)** until the user presses **Enter (0Dh)**.
The conversion is done in three steps:

1. **ASCII to digit:** Each character is turned into a number using SUB AL, '0'.
2. **Build the full number:** The previous total is multiplied by 10, and the new digit is added (e.g., 4 → 40 → 47).
3. **Store the result:** When Enter is pressed, the final numeric value in **AL** is stored in the array with MOV [ARRAY + SI], AL.

This prepares the data for the bubble sort routine.

The Bubble Sort Logic:

The Bubble Sort is implemented using two nested loops to handle the passes and comparisons:

1. Outer Loop (OUTLUP)

- Controls the number of passes through the array.
- `CL` is set to **N–1**, and after each pass, `CL` is decremented.
- The loop continues while `CL ≠ 0` (`JNZ OUTLUP`).

2. Inner Loop (INLUP)

- Performs comparisons within each pass.
- `BH` stores how many comparisons remain in the current pass.
- Each iteration compares `[SI]` and `[SI+1]`, then moves to the next pair.

3. Swap Block

- Happens only if the current value is greater than the next value.
- A simple 3-step swap is done using **DL** as a temporary register.

This repeated comparison and swapping eventually results in a fully sorted array.

Output:

Conversion Logic:

To display the numeric byte value from the array, the procedure converts the number back into ASCII digits using repeated Division by 10:

- The number in AL is divided by 10 (DIV BL, where BL=10). The Remainder (AH) is the rightmost digit.
- The remainder is converted to ASCII by adding 30h (ADD AH, 30h).
- The digits are stored backward in the DIGIT_BUFFER as the loop continues.

Output:

After conversion, the routine prints the string stored in DIGIT_BUFFER using INT 21h, AH=09h, followed by a space separator (INT 21h, AH=02h).

Service Routines and Procedures Used:

- **INT 21h Functions Used**

Purpose	Interrupt	Description
Display string	INT 21h, AH=09h	Prints $-terminated string
Display character	INT 21h, AH=02h	Prints single character in DL
Read a character	INT 21h, AH=01h	Reads a keystroke (for digit input)
Exit program	INT 21h, AH=4Ch	Returns control to DOS

- **Procedures Used:**
  1. ***read_and_convert_byte* Procedure**
     - Reads up to 3 digits individually using INT 21h/01h
     - Validates digits
     - Converts ASCII → integer
     - Handles overflow (>255)
     - Returns final integer in AL

  2. ***byte_to_ascii_print* Procedure**
     - Converts a byte (0–255) into ASCII digits
     - Stores digits inside digit_buffer
     - Uses repeated division by 10 to extract digits

- Prints using INT 21h/09h

Justification for Each Structure and Routine

Why use an Array?

- Bubble sort requires random access; arrays allow direct indexing ([si]).
- Efficient way to store multiple values for iterative processing.

Why use Loops?

- Sorting and printing require repeated operations.
- LOOP instruction simplifies iteration.

Why use Conditional Jumps?

- Bubble sort compares and swaps values.
- Conditional jumps (JBE, JL, JE) allow branching based on comparisons.

Why use INT 21h routines?

- Provide simple and standardized input/output for DOS.
- No need to handle hardware ports manually.
- Makes reading input and displaying strings much easier.

Why use Stack (PUSH/POP)?

- Required when writing modular procedures.
- Prevents overwriting registers used by main code.
- Ensures each procedure is self-contained.

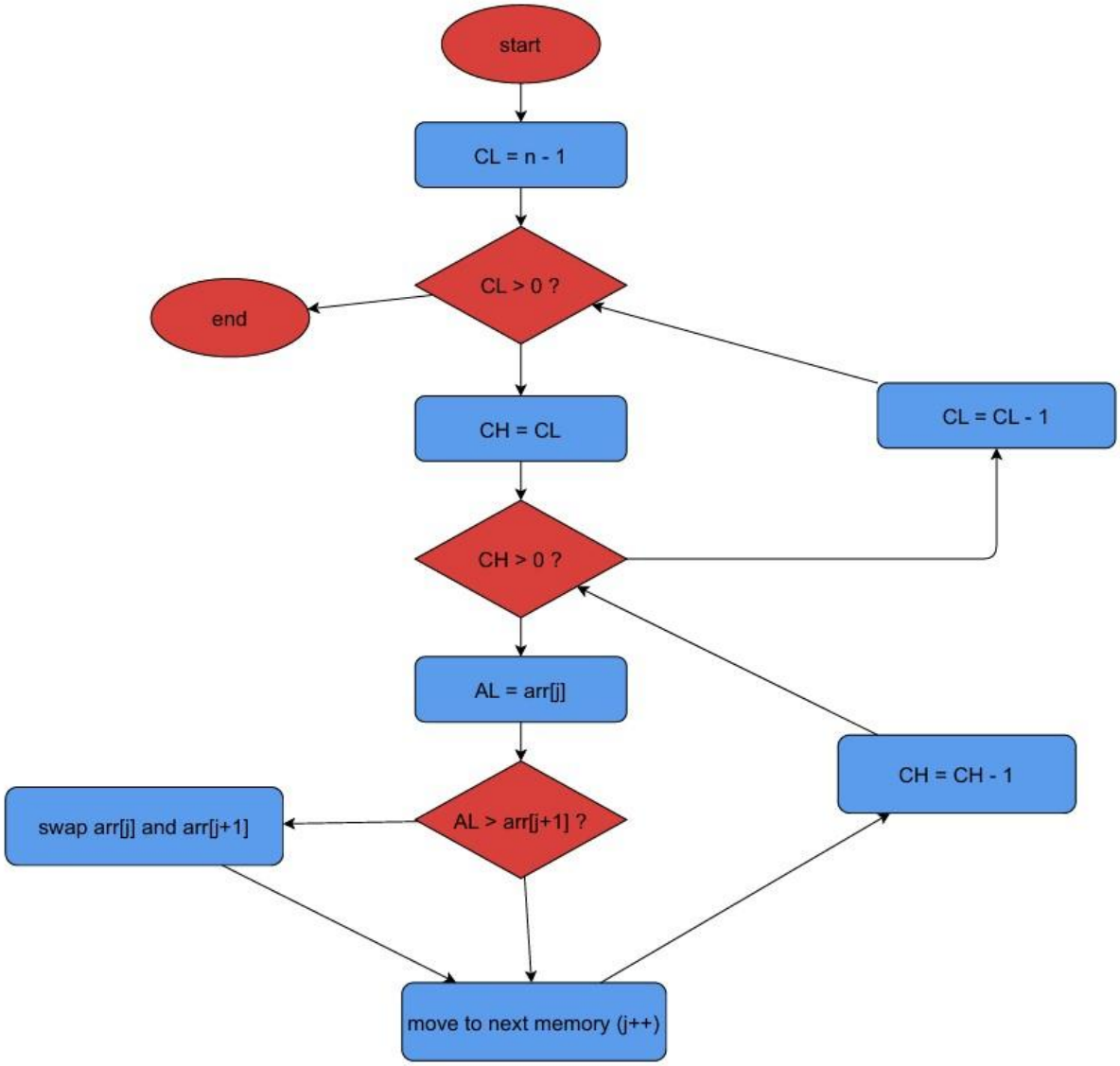
Why separate input and printing into procedures?

- Improves readability and reusability.
- Avoids code duplication.
- Makes debugging easier.

Flow Chart:

Explanation:

- Outer loop runs N–1 time
- Inner loop compares each adjacent pair
- Swap if the earlier number is bigger
- Each pass pushes the biggest element to the end
- Eventually all numbers become sorted



Conclusion

This project successfully demonstrates several key concepts of Assembly Language and Computer Organization:

- Direct register manipulation (AX, BX, SI, DI, CX, DX)
- Use of DOS interrupt services for input/output
- Memory addressing and array handling
- Looping and conditional branching
- Stack-based procedure calling

- Number-to-ASCII and ASCII-to-number conversion
- Implementation of a standard algorithm (Bubble Sort) entirely in low-level code

This project demonstrates a solid understanding of Assembly Language, modular design, arithmetic operations, and registers.